

StreamFlow

Sistema de gestión de contenidos multimedia en Java

Integrantes: Santiago Castillo, Darío Minga

Docente: Ingeniero Bustamante Granda Wayner Xavier

Carrera: Ingeniería en computación

1. ¿Para qué sirve el proyecto?

StreamFlow es una aplicación de escritorio desarrollada en Java (ejecutada por consola) que simula el funcionamiento interno de una plataforma de streaming, similar en concepto a Netflix. Permite administrar un catálogo de contenidos audiovisuales (películas, series y documentales), gestionar usuarios, calcular el costo de una suscripción según la calidad elegida y generar recomendaciones simples de contenido según el género.

El proyecto no cuenta con interfaz gráfica: toda la interacción se realiza mediante un menú de texto que se muestra en la consola, y cada acción del usuario (agregar, listar, buscar, eliminar) se traduce en operaciones sobre una base de datos SQLite almacenada en el propio proyecto (streamflow.db).

Funcionalidades principales

- Gestión de contenidos: registrar, listar, buscar por ID y eliminar películas, series o documentales.
- Gestión de usuarios: registrar, listar, buscar y eliminar usuarios del servicio.
- Cálculo de suscripción: determina el plan (Básico, Estándar o Premium) y el precio mensual según la calidad de video elegida (SD, HD o 4K).
- Recomendaciones: sugiere contenidos del catálogo que coincidan con un género indicado por el usuario.

2. ¿Cómo funciona?

El programa se ejecuta desde la clase Main, que simplemente crea el menú principal y lo pone en marcha. A partir de ahí, todo el flujo se maneja mediante un ciclo de menús por consola:

Opción del menú	Acción que realiza
1. Gestionar contenidos	Submenú para listar, agregar, buscar o eliminar películas, series y documentales.
2. Gestionar usuarios	Submenú para listar, agregar, buscar o eliminar usuarios.
3. Calcular suscripción	Pide la calidad deseada y muestra el plan y el costo mensual.
4. Recomendar contenidos	Pide un género y muestra los contenidos del catálogo que coinciden.
0. Salir	Termina el programa.

Cada vez que el usuario elige agregar un contenido, el programa primero pregunta el tipo (película, serie o documental) y luego pide únicamente los datos propios de ese tipo (por ejemplo, director y año para una película, o temporadas y capítulos para una serie). Con esos datos se construye un objeto en memoria y se envía a la base de datos para guardarlo de forma permanente.

Internamente, la aplicación sigue el patrón de arquitectura por capas (similar a MVC), donde cada capa tiene una responsabilidad clara:

Capa	Paquete	Responsabilidad
Vista	Vista	Muestra los menús y captura lo que escribe el usuario en consola.

Capa	Paquete	Responsabilidad
Controlador	Controlador	Recibe la petición de la vista y decide a qué parte del sistema pedirle los datos.
Servicio	Servicio	Contiene reglas de negocio: cálculo de suscripción y lógica de recomendaciones.
Modelo	Modelo	Representa los objetos del sistema: Contenido (y sus tipos), Usuario, Calidad.
Acceso a datos (DAO)	DAO	Traduce los objetos del Modelo en instrucciones SQL y viceversa.
Base de datos	Database	Abre y administra la conexión con el archivo streamflow.db (SQLite).

Gracias a esta separación, la Vista nunca habla directamente con la base de datos: siempre pasa primero por el Controlador, que a su vez usa el Servicio o el DAO según lo que necesite. Esto hace que el código sea más ordenado y fácil de modificar.

3. Datos importantes del código

3.1 Tecnologías utilizadas

Aspecto	Detalle
Lenguaje	Java 17
Entorno de desarrollo	NetBeans (proyecto con build.xml / Ant)
Base de datos	SQLite, mediante el driver sqlite-jdbc 3.36.0.3
Tipo de aplicación	Consola (sin interfaz gráfica)
Clase de entrada	com.streamflow.Main

3.2 Estructura de paquetes

```
src/
├── com/StreamFlow/Main.java           (punto de entrada del programa)
├── Vista/MenuPrincipal.java           (menús por consola)
├── Controlador/
│   ├── ContenidoController.java
│   └── UsuarioController.java
├── Servicio/
│   ├── SuscripcionService.java
│   └── RecomendacionService.java
├── Modelo/
│   ├── Contenido.java   (clase abstracta base)
│   ├── Pelicula.java
│   ├── Serie.java
│   ├── Documental.java
│   ├── Usuario.java
│   └── Calidad.java     (enum: SD, HD, 4K)
├── DAO/
│   └── IContenidoDAO.java / ContenidoDAO.java
```



3.3 Patrones de diseño aplicados

El código está pensado para ser fácilmente ampliable en el futuro, y por eso aplica varios patrones de diseño conocidos:

- Herencia y polimorfismo: Contenido es una clase abstracta de la que heredan Pelicula, Serie y Documental. Cada una implementa a su manera los métodos reproducir() y obtenerDetalles(), y declara su propio tipo mediante getTipo().
- Factory (fábrica): ContenidoFactory se encarga de crear el objeto correcto (Pelicula, Serie o Documental) a partir del texto "tipo" que viene de la base de datos, evitando así tener que usar instanceof o largas cadenas de if/else.
- DAO (Data Access Object): ContenidoDAO y UsuarioDAO separan por completo la lógica SQL del resto del programa. Cada uno implementa una interfaz (IContenidoDAO / IUsuarioDAO), lo que permite reemplazar la forma de guardar los datos sin tener que tocar el resto del sistema.
- Principio Abierto/Cerrado (OCP): al usar la Factory y el método getTipo() de cada subclase, es posible agregar un nuevo tipo de contenido (por ejemplo "Documental interactivo") sin modificar el código ya existente del DAO.
- Enum con comportamiento: Calidad no es solo una lista de valores (SD, HD, 4K); cada valor lleva asociado su propio precio, usado directamente por el servicio de suscripción.

3.4 Clases más relevantes

Clase	Qué hace
Contenido (abstracta)	Define los atributos comunes a todo contenido: ID, título, género y calidad, más los métodos que cada subtipo debe implementar.
Pelicula / Serie / Documental	Extienden Contenido y agregan sus propios atributos (duración y director; temporadas y capítulos; tema, respectivamente).
Calidad (enum)	Define las calidades disponibles (SD, HD, 4K) y el precio mensual asociado a cada una.
Usuario	Representa a un usuario del sistema (ID, nombre y plan contratado).
ContenidoFactory	Crea instancias de Pelicula, Serie o Documental según el tipo indicado, sin exponer los detalles de construcción.
ContenidoDAO / UsuarioDAO	Ejecutan las sentencias SQL (INSERT, SELECT, UPDATE, DELETE) usando PreparedStatement para evitar inyección SQL.
ConexionSQLite	Abre la conexión al archivo streamflow.db y crea las tablas si todavía no existen.
SuscripcionService	Calcula el precio y el nombre del plan según la calidad elegida.
RecomendacionService	Filtra la lista de contenidos y devuelve solo los que coinciden con el género buscado.
MenuPrincipal	Controla todo el flujo de menús y submenús que ve el usuario en la consola.

4. Explicación de la base de datos

StreamFlow utiliza SQLite, un motor de base de datos ligero que guarda toda la información en un único archivo (streamflow.db), sin necesidad de instalar un servidor de base de datos aparte. El acceso se realiza mediante JDBC (driver sqlite-jdbc), y la conexión se centraliza en la clase ConexionSQLite.

4.1 Tablas creadas automáticamente por el programa

Cada vez que la aplicación se conecta por primera vez a la base de datos, revisa si las tablas principales existen y, si no, las crea. Estas son las dos tablas que el código Java gestiona de forma activa:

Tabla	Columnas	Descripción
Contenido	id_contenido (PK), titulo, genero, calidad, tipo	Guarda todos los contenidos del catálogo (películas, series y documentales) en una sola tabla; la columna "tipo" indica de qué clase se trata.
Usuario	id_usuario (PK), nombre, plan	Guarda los usuarios registrados y el plan de suscripción que tienen asignado.

Esta es una estrategia de "tabla única por jerarquía" (single table inheritance): en lugar de crear una tabla distinta para Película, Serie y Documental, toda la información común se guarda en la tabla Contenido, y es la clase ContenidoFactory la que, al leer una fila, reconstruye el objeto Java correcto según el valor de la columna tipo.

4.2 Tablas adicionales presentes en el archivo .db

El archivo streamflow.db incluido en el proyecto contiene, además, un diseño más completo que el que el código Java crea automáticamente. Esto sugiere un modelo relacional más detallado, pensado para una versión más avanzada del sistema, aunque el código actual todavía no interactúa con estas tablas adicionales:

Tabla	Columnas principales	Propósito previsto
Pelicula	id_contenido (PK/FK), duracion, director, anio_estreno	Guardaría los datos específicos de una película en una tabla separada, relacionada 1 a 1 con Contenido.
Serie	id_contenido (PK/FK), temporadas, capitulos, duracion_capitulo	Guardaría los datos específicos de una serie, relacionada 1 a 1 con Contenido.
Documental	id_contenido (PK/FK), tema, duracion	Guardaría los datos específicos de un documental, relacionada 1 a 1 con Contenido.
Suscripcion	id_suscripcion (PK), id_usuario (FK), plan, precio	Registraría el historial de suscripciones contratadas por cada usuario.
Historial	id_historial (PK), id_usuario (FK), id_contenido (FK)	Registraría qué contenidos ha visto cada usuario (historial de reproducción).

En otras palabras, el archivo de base de datos ya está preparado con claves foráneas (FOREIGN KEY) hacia Contenido y Usuario para soportar estas relaciones, pero por ahora solo las tablas Contenido y Usuario reciben datos desde el código Java. Esto es normal en proyectos académicos en desarrollo: la base de datos se diseña pensando en

el crecimiento futuro del sistema (por ejemplo, guardar el historial de reproducción o el historial de pagos de cada usuario).

4.3 Relaciones entre tablas

- Usuario (1) — (N) Suscripcion: un usuario puede tener varias suscripciones a lo largo del tiempo.
- Usuario (1) — (N) Historial: un usuario puede tener muchos registros de contenidos vistos.
- Contenido (1) — (N) Historial: un mismo contenido puede aparecer en el historial de muchos usuarios.
- Contenido (1) — (1) Pelicula / Serie / Documental: cada contenido "especializado" amplía la fila general de Contenido con sus propios atributos.

4.4 Cómo se comunica el código con la base de datos

El flujo típico para guardar un dato (por ejemplo, un nuevo contenido) es el siguiente:

1. La Vista (MenuPrincipal) recoge los datos que escribe el usuario.
2. El Controlador (ContenidoController) recibe el objeto ya construido y lo pasa al DAO.
3. El DAO (ContenidoDAO) arma una sentencia SQL con PreparedStatement (para evitar inyección SQL) y la ejecuta.
4. La clase ConexionSQLite abre la conexión al archivo streamflow.db y, si es la primera vez, crea las tablas necesarias.
5. SQLite guarda el cambio directamente en el archivo streamflow.db, sin necesidad de un servidor externo.

Para leer datos (por ejemplo, listar el catálogo), el proceso es igual pero en sentido inverso: el DAO ejecuta un SELECT, recorre las filas obtenidas y usa ContenidoFactory para reconstruir cada objeto Java (Pelicula, Serie o Documental) antes de devolverlos a la Vista.

5. Análisis de los principios SOLID aplicados

A continuación se analiza, principio por principio, en qué medida el código de StreamFlow los cumple y dónde existen oportunidades de mejora.

S — Principio de Responsabilidad Única (SRP)

Cada clase tiene, en general, una única razón para cambiar: MenuPrincipal solo se ocupa de mostrar menús y leer la entrada del usuario; ContenidoController y UsuarioController solo coordinan la petición entre la Vista y el DAO; ContenidoDAO y UsuarioDAO solo saben traducir objetos a SQL y viceversa; SuscripcionService solo calcula precios y planes; RecomendacionService solo filtra contenidos por género. Como observación menor, MenuPrincipal concentra los cuatro submenús de la aplicación (contenidos, usuarios, suscripción, recomendaciones) en una sola clase; en un proyecto de mayor tamaño convendría dividirla en una clase de vista por cada módulo.

O — Principio Abierto/Cerrado (OCP)

Este es uno de los principios mejor logrados en el proyecto. Gracias a ContenidoFactory y a que cada subclase de Contenido declara su propio tipo mediante getTipo(), es posible agregar un nuevo tipo de contenido (por ejemplo, "Cortometraje") registrándolo en la fábrica, sin modificar ni una línea de ContenidoDAO. El DAO está "cerrado a modificación" pero "abierto a extensión", que es exactamente la idea del principio.

L — Principio de Sustitución de Liskov (LSP)

Película, Serie y Documental pueden usarse en cualquier lugar donde se espera un Contenido sin alterar el comportamiento correcto del programa: los métodos reproducir() y obtenerDetalles() están sobrescritos de forma coherente con el contrato de la clase base, y ninguna subclase restringe ni contradice lo que promete Contenido. Por ejemplo, listarContenidos() en MenuPrincipal trata a cualquier subtipo de manera uniforme, sin necesidad de preguntar de qué clase concreta se trata.

I — Principio de Segregación de Interfaces (ISP)

En lugar de definir una única interfaz genérica de "persistencia", el proyecto separa IContenidoDAO e IUserDAO, cada una con solo los métodos que su respectivo cliente necesita (guardar, listar, buscar, actualizar, eliminar). Ningún controlador se ve obligado a implementar o depender de métodos que no usa.

D — Principio de Inversión de Dependencias (DIP)

Los controladores (ContenidoController, UsuarioController) dependen de las interfaces IContenidoDAO / IUserDAO y no de las clases concretas, e incluso ofrecen un constructor que permite inyectar una implementación distinta (por ejemplo, un DAO de prueba en memoria), lo cual es una aplicación correcta del principio. La excepción es la clase ConexionSQLite: los DAO la referencian de forma estática y directa, por lo que, en sentido estricto, ese punto concreto del código depende de una implementación concreta (SQLite) en lugar de una abstracción. No es un problema grave para un proyecto de este tamaño, pero sería el siguiente paso de mejora si se quisiera, por ejemplo, cambiar de motor de base de datos.

Resumen

Principio	Nivel de cumplimiento	Observación clave
S — Responsabilidad única	Alto	MenuPrincipal podría dividirse en submódulos más pequeños.
O — Abierto/Cerrado	Alto	Logrado mediante ContenidoFactory y getTipo().
L — Sustitución de Liskov	Alto	Las subclases de Contenido son intercambiables sin romper el programa.
I — Segregación de interfaces	Alto	Interfaces separadas y específicas por tipo de entidad.
D — Inversión de dependencias	Medio-alto	Los DAO dependen directamente de ConexionSQLite (SQLite) en vez de una abstracción.

6. Evidencias de ejecución

Este informe documenta el análisis del código y de la base de datos a partir de los archivos del proyecto. Sin embargo, no incluye capturas de pantalla de la ejecución ni un reporte de JUnit en verde, por dos razones concretas que es importante conocer:

- El proyecto no cuenta actualmente con clases de prueba: la carpeta test/ del proyecto está vacía y no tiene la librería JUnit agregada en NetBeans. No existe, por tanto, ningún reporte de JUnit que se pueda mostrar.

- Generar capturas de pantalla o un reporte "en verde" sin haber ejecutado realmente las pruebas implicaría presentar evidencia falsa, algo que este informe evita de forma deliberada.

Para que usted pueda obtener esta evidencia de forma legítima, se recomienda el siguiente procedimiento:

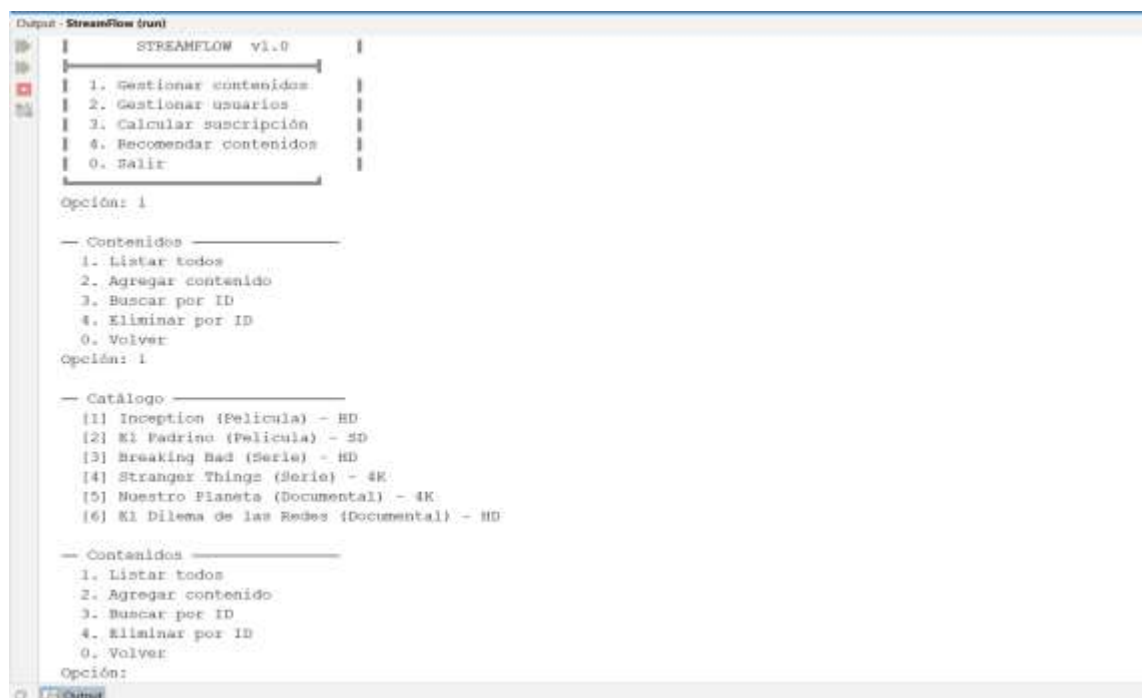
1. En NetBeans, clic derecho sobre el proyecto → Properties → Libraries → pestaña Test Libraries → Add Library → JUnit (versión 5 recomendada).
2. Crear las clases de prueba dentro de la carpeta test/, siguiendo el mismo paquete que la clase que se está probando (por ejemplo, Servicio/SuscripcionServiceTest.java).
3. Ejecutar las pruebas con clic derecho sobre el proyecto → Test, o sobre un archivo de prueba → Test File.
4. Tomar la captura de pantalla del panel de resultados de NetBeans (o del IDE que use) cuando las pruebas terminen en verde, y capturas del programa en ejecución (consola) al usar cada opción del menú.

Como apoyo adicional, se generó un archivo StreamFlowTest.java con casos de prueba reales para las clases de lógica de negocio del proyecto (SuscripcionService, RecomendacionService y ContenidoFactory), listo para copiar dentro de la carpeta test/ del proyecto y ejecutar en NetBeans para producir el reporte de JUnit genuino.

7. Conclusión

StreamFlow es un proyecto que reproduce, a pequeña escala, el funcionamiento de una plataforma de streaming: administra contenidos y usuarios, calcula suscripciones y ofrece recomendaciones simples, todo mediante una interfaz de consola respaldada por una base de datos SQLite. Su principal fortaleza es el uso de buenas prácticas de diseño en Java (herencia, patrón Factory, patrón DAO, separación en capas y un alto cumplimiento de los principios SOLID), lo que facilita entender el código y ampliarlo más adelante, por ejemplo, conectando las tablas Suscripcion e Historial que ya existen en la base de datos, o incorporando pruebas unitarias con JUnit.

8. Imágenes de programa




```

Opción: 0

┌───────────────────┐
│ STREAMFLOW  v1.0  │
├───────────────────┤
│ 1. Gestionar contenidos  │
│ 2. Gestionar usuarios    │
│ 3. Calcular suscripción  │
│ 4. Recomendar contenidos │
│ 0. Salir                 │
└───────────────────┘

Opción: 2

— Usuarios —
1. Listar todos
2. Agregar usuario
3. Buscar por ID
4. Eliminar por ID
0. Volver

Opción: 1

— Usuarios —
[1] Carlos Perez — Plan: HD
[2] Ana Torres — Plan: 4K
[3] Luis Mora — Plan: SD

— Usuarios —
1. Listar todos
2. Agregar usuario
3. Buscar por ID
4. Eliminar por ID
0. Volver

Opción:
Output

```

```

Jupyter - StreamFlow (run)

— Usuarios —
1. Listar todos
2. Agregar usuario
3. Buscar por ID
4. Eliminar por ID
0. Volver

Opción: 0

┌───────────────────┐
│ STREAMFLOW  v1.0  │
├───────────────────┤
│ 1. Gestionar contenidos  │
│ 2. Gestionar usuarios    │
│ 3. Calcular suscripción  │
│ 4. Recomendar contenidos │
│ 0. Salir                 │
└───────────────────┘

Opción: 4
Género a buscar: Terror

— Recomendaciones para: Terror —
[4] Stranger Things (Serie)

```

Output - StreamFlow (run)

```

STREAMFLOW v1.0
1. Gestionar contenidos
2. Gestionar usuarios
3. Calcular suscripción
4. Recomendar contenidos
0. Salir

Opción: 3
Calidad: 1=SD 2=HD 3=4K
Calidad: 1

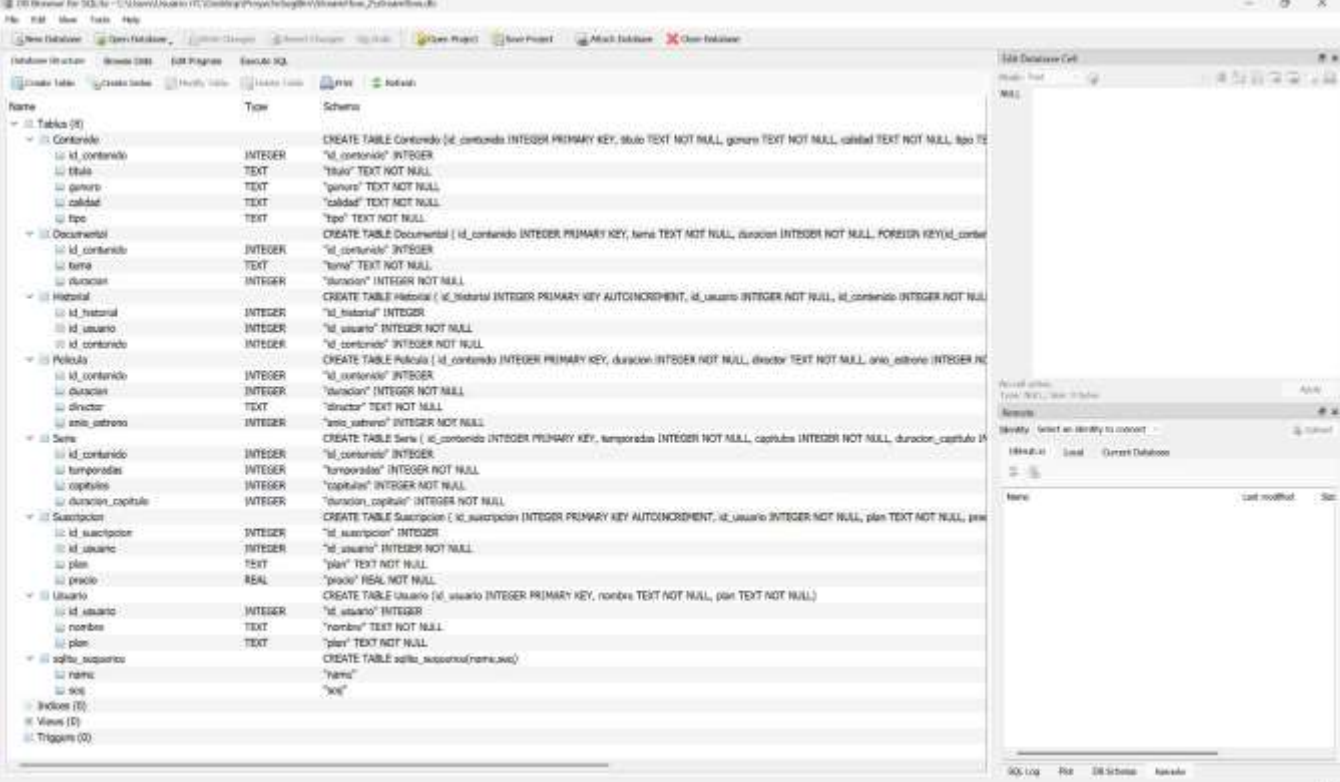
Plan: Plan Básico (SD) - $5,00 / mes

STREAMFLOW v1.0
1. Gestionar contenidos
2. Gestionar usuarios
3. Calcular suscripción
4. Recomendar contenidos
0. Salir

Opción: 3
Calidad: 1=SD 2=HD 3=4K
Calidad: 2

Plan: Plan Estándar (HD) - $8,00 / mes

```



The screenshot shows the DBeaver SQL Editor interface with a database schema for 'StreamFlow'. The schema is organized into a tree view on the left, showing tables and their relationships. The main editor area displays the SQL DDL for each table, including primary keys, foreign keys, and data types.

Table Name	Columns	Data Types	Constraints
Contenido	id_contenido, titulo, genero, calidad, tipo	INTEGER, TEXT	PRIMARY KEY (id_contenido)
Documental	id_contenido, tema, duracion	INTEGER, TEXT	PRIMARY KEY (id_contenido), FOREIGN KEY (id_contenido) to Contenido
Historial	id_historial, id_usuario, id_contenido	INTEGER	PRIMARY KEY (id_historial), FOREIGN KEY (id_usuario) to Usuario, FOREIGN KEY (id_contenido) to Contenido
Pelicula	id_contenido, duracion, director, emisor_estreno	INTEGER, TEXT	PRIMARY KEY (id_contenido), FOREIGN KEY (id_contenido) to Contenido
Serie	id_contenido, temporadas, capitulos, duracion_capitulo	INTEGER	PRIMARY KEY (id_contenido), FOREIGN KEY (id_contenido) to Contenido
Suscripcion	id_suscripcion, id_usuario, plan, precio	INTEGER, TEXT, REAL	PRIMARY KEY (id_suscripcion), FOREIGN KEY (id_usuario) to Usuario
Usuario	id_usuario, nombre, plan	INTEGER, TEXT	PRIMARY KEY (id_usuario)
Tabla Suscripcion	id_usuario, id_suscripcion, nombre_usuario	INTEGER	PRIMARY KEY (id_usuario, id_suscripcion), FOREIGN KEY (id_usuario) to Usuario, FOREIGN KEY (id_suscripcion) to Suscripcion